

Lab: 0100

Adder Subtractor, Two's Complement & Overflow

CSA222: Modern Computer Architecture

4-BIT ADDER-SUBTRACTOR

OPERATION

K = 0 (ADDITION)

- $B' = B$
- $Cin = 0$
- Result = $A + B$

K = 1 (SUBTRACTION)

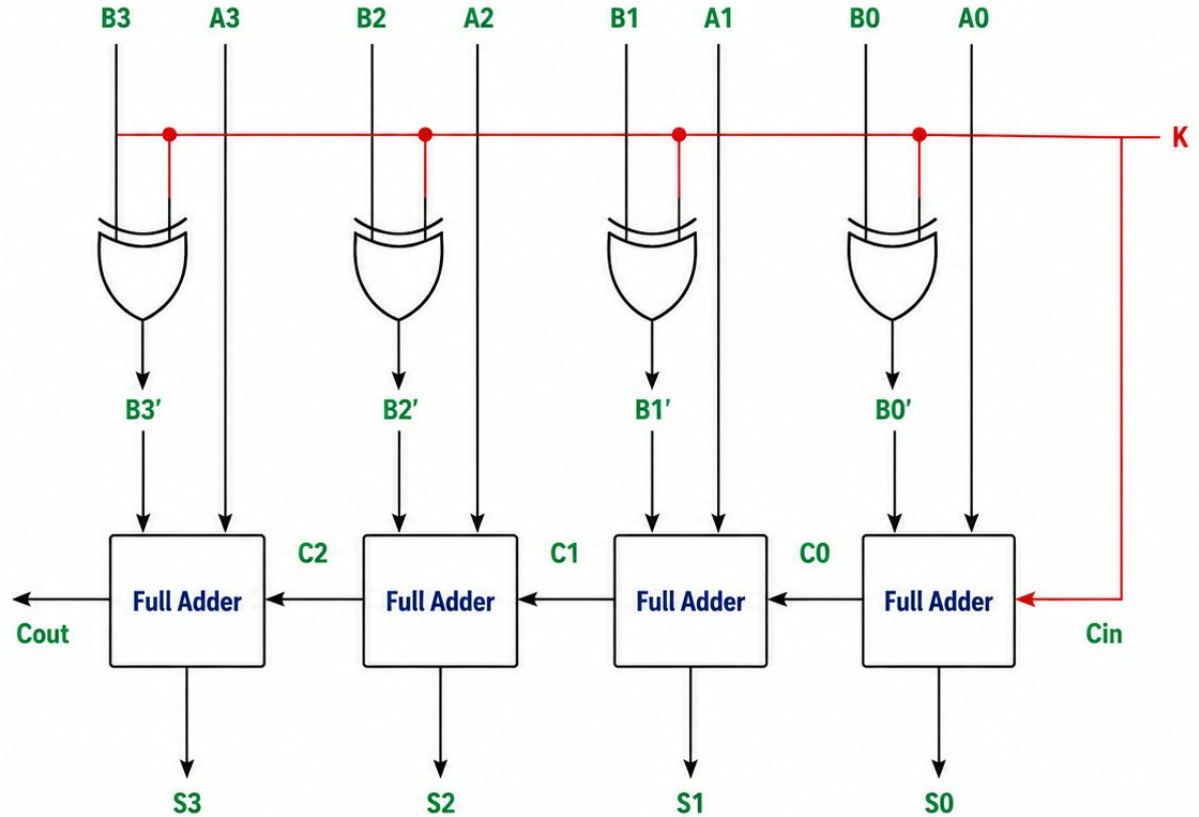
- $B' = \sim B$
- $Cin = 1$
- Result = $A + (\sim B) + 1 = A - B$

HOW IT WORKS

- Each bit of B is XORed with control K.
- When $K = 0$, B remains unchanged.
- When $K = 1$, B is inverted.
- K also acts as the initial carry (Cin).
- The same adder hardware performs both addition and subtraction.

SIGNALS

- A[3:0]** → First operand
B[3:0] → Second operand
K → Mode select (0 = Add, 1 = Sub)
S[3:0] → Result
Cout → Carry-out



```
module adder_subtractor(  
    input  [3:0] a,  
    input  [3:0] b,  
    input           m,  
    output [3:0] s,  
    output          cout  
);  
  
wire [3:0] b_in;  
assign b_in = b ^ {4{m}};           // XOR each bit of b with m  
assign {cout, s} = a + b_in + m;    // Add a, modified b and m (Cin)  
  
endmodule
```

Can you Try 1 & 2 bit Comparator and Multiplier by yourself ?

Explore Carry Look Ahead Adder and try to figure out something which is worth noticing !

Two's Complement • The Lay of the Land

Same 4 bits. Two readings. Whether a result is 'right' depends entirely on which reading you intended.

U

UNSIGNED • 0 to 15

All bits are 'just bits'. 1111 = 15. Carry-out = wraparound. $15 + 1 \rightarrow 0 + \text{Cout}$. That's it. Cout IS the overflow flag.

S

SIGNED (TWO'S COMPLEMENT) • -8 to +7

MSB is the sign bit. 1111 = -1, not 15. Cout is meaningless here — we need a different flag.

BITS	0000	0001	...	0111	1000	1001	1111
UNSIGNED	0	1	...	7	8	9	15
SIGNED	0	1	...	7	-8	-7	-1

Unsigned Overflow • CF = Cout

If you treat your 4 bits as 0–15, the carry-out tells you the result didn't fit. End of story.

CF (carry flag) = Cout

0001 = 1
+0001 = 1
0010 = 2

CF = 0

Result fits in 4 bits. No overflow.

1111 = 15
+0001 = 1
0000 = 0

CF = 1

15+1 = 16, but only 4 bits — the top bit fell off.

1000 = 8
+1000 = 8
0000 = 0

CF = 1

8+8 = 16 = 10000. Bit 4 became Cout.

Signed Overflow · $OF = C_3 \oplus C_4$

The XOR of the carry INTO the MSB and the carry OUT of the MSB. That's the rule, and it just works.

OF (overflow flag) = Cin-to-MSB $\oplus$ Cout-of-MSB

THE INTUITION

Signed overflow happens when two positives add to a 'negative', or two negatives add to a 'positive' — the sign bit lies about the answer.

The MSB column is exactly where that lie happens. If the carry going IN disagrees with the carry going OUT, the sign bit got flipped by arithmetic — not by the real result. That's overflow.

WHERE TO TAP · In your CLA, c_3 is the carry into the MSB and c_4 (= cout) is the carry out. So $OF = c_3 \oplus c_4$.

Why That XOR Works

Two signs in, one sign out. Walk the MSB column. Overflow is exactly the disagreement of its two carries.

A_3	B_3	Result sign	C_3 in	C_4 out	OF	Verdict
0 (+)	0 (+)	0 ✓ matches	0	0	0	no overflow
0 (+)	0 (+)	1 ✗ flipped	1	0	1	OVERFLOW
1 (-)	1 (-)	0 ✗ flipped	0	1	1	OVERFLOW
1 (-)	1 (-)	1 ✓ matches	1	1	0	no overflow

READ MIDDLE TWO ROWS · Same-sign inputs producing the opposite sign ⇔ the MSB column's two carries disagree.

Overflow Flags • Two More Lines

Promote c3 and c4 to module outputs. One AND, one XOR, and you're done.

```
cla_adder.v • with flags
module cla_adder_4bit_ovf (
    input wire [3:0] a, b,
    input wire cin,
    output wire [3:0] sum,
    output wire cout, cf, of
);
    // ... g, p, c1..c4, sum exactly as before ...

    // The two new flags:
    assign cf = c4;           // unsigned overflow
    assign of = c3 ^ c4;      // signed overflow
    assign cout = c4;
endmodule
```

TWO FLAGS, ONE ADDER

cf is meaningful only if you read your inputs as unsigned.

of is meaningful only if you read them as signed.

The adder emits both flags — your program (or CPU flag register) picks. Exactly how x86 and ARM work.

Four Test Cases · Stressing Both Flags

Each case tests a different signed/unsigned scenario. Predict every flag before you simulate.

01 0001 + 0001

CF=0 OF=0

unsigned 1+1=2 · signed +1+1=+2 · quiet case

02 0111 + 0001

CF=0 OF=1

unsigned 7+1=8 (fine) · signed +7+1=-8 ✗

03 1111 + 0001

CF=1 OF=0

unsigned 15+1=16 ✗ wrapped · signed -1+1=0 (fine)

04 1000 + 1000

CF=1 OF=1

unsigned 8+8=16 ✗ · signed -8+-8=0 ✗ · both broke

Drop these four into your testbench. Print a, b, sum, cf, of for each — the pattern above is the spec.

Walking Through Test Cases · Part 1

Trace the MSB column by hand. If sim disagrees, your flag logic is wrong — not the arithmetic.

case 02

0111 + 0001 · signed overflow

0111 = +7

+ 0001 = +1

1000 = -8 (should be +8)

MSB: Cin=1 Cout=0

OF = 1 xor 0 = 1

CF = Cout = 0

case 03

1111 + 0001 · unsigned wrap

1111 = 15 (or -1)

+ 0001 = +1

0000 = 0

MSB: Cin=1 Cout=1

OF = 1 xor 1 = 0

CF = Cout = 1

Walking Through Test Cases · Part 2

The double-failure case, plus a 'should-be-clean' case to show what a quiet adder looks like.

case 04

1000 + 1000 · both flags

1000 = 8 (or -8)

+ 1000 = 8 (or -8)

0000 = 0

MSB: Cin=0 Cout=1

OF = 0 xor 1 = 1

CF = Cout = 1

sanity check

0011 + 0010 · the quiet case

0011 = +3

+ 0010 = +2

0101 = +5

MSB: Cin=0 Cout=0

OF = 0 xor 0 = 0

CF = Cout = 0

END OF LAB 04

**One adder performs all.
One ground truth.
Two flags for what 'overflow'
really means.**



**Thanks
for
watching!**